E21 — Algoritmo de ordenamiento Bubble Sort

🎯 Objetivo del ejercicio

- Comprender el funcionamiento del algoritmo de ordenamiento **Bubble Sort**.
- Analizar cómo se realizan las comparaciones e intercambios entre elementos.
- Comparar la versión tradicional del algoritmo con una versión optimizada usando **variable bandera**.

📘 Teoría y conceptos

Bubble Sort es un algoritmo de ordenamiento sencillo que compara pares de elementos adyacentes y los intercambia si están en el orden incorrecto.
Su funcionamiento básico:

1. Recorrer la lista varias veces.
2. En cada pasada, comparar elementos consecutivos.
3. Intercambiar si es necesario.
4. Repetir hasta que la lista esté ordenada.

El algoritmo tradicional **siempre recorre toda la lista**, incluso si ya está ordenada antes de terminar todas las pasadas.
Esto genera:

- Iteraciones innecesarias
- Tiempo de ejecución mayor
- Ineficiencia en listas casi ordenadas

Una variable bandera (por ejemplo, hubo_cambios) permite detectar si en una pasada **se hizo al menos un intercambio**.
Si en una pasada **no hubo cambios**, significa que la lista ya está ordenada y el algoritmo puede detenerse antes.
Ventajas:

- Reduce iteraciones
- Acelera el ordenamiento
- Mejora el rendimiento en listas parcialmente ordenadas

📝 Instrucciones

Parte 1 — Bubble Sort tradicional

1. Teclea el código en **Python Tutor**.
2. Ejecuta paso a paso (Next → Next → Next).
    - Observa: Cómo se comparan los elementos, cuántas veces se recorre la lista y cuántos intercambios se realizan.
3. Prueba con listas pequeñas y luego con listas casi ordenadas.

Parte 2 — Bubble Sort optimizado

1. Teclea el segundo código en **Python Tutor**.
2. Ejecuta paso a paso.
    - Observa: Cómo la bandera detiene el algoritmo antes y aué pasa cuando la lista ya está ordenada.
3. Compara el número de iteraciones con la versión anterior.

Código 1 — Bubble Sort tradicional

```
lista = [5, 1, 4, 2, 8] #Edita esto para hacer tus pruebas
print("Lista original:", lista)
n = len(lista)
# Recorrer la lista n-1 veces
for i in range(n - 1):
    print("\nPasada número:", i + 1)
    # Comparar elementos adyacentes
    for j in range(n - 1):
        print("  Comparando", lista[j], "y", lista[j + 1])
        if lista[j] > lista[j + 1]:
            print("   -> Intercambio")
            lista[j], lista[j + 1] = lista[j + 1], lista[j]
    print("  Lista después de la pasada:", lista)
print("\nLista ordenada:", lista)
```

Código 2 — Bubble Sort optimizado (con variable bandera)

```
lista = [5, 1, 4, 2, 8] #Edita esto para hacer tus pruebas
print("Lista original:", lista)
n = len(lista)
for i in range(n - 1):
    print("\nPasada número:", i + 1)
    hubo_cambios = False   # Bandera para detectar intercambios
    for j in range(n - 1):
        print("  Comparando", lista[j], "y", lista[j + 1])
        if lista[j] > lista[j + 1]:
            print("   -> Intercambio")
            lista[j], lista[j + 1] = lista[j + 1], lista[j]
            hubo_cambios = True
    print("  Lista después de la pasada:", lista)
    # Si no hubo cambios, la lista ya está ordenada
    if not hubo_cambios:
        print("No hubo intercambios. La lista ya está ordenada.")
        break
print("\nLista ordenada:", lista)
```

Entregable por equipo

Cada equipo (1–3 personas) deberá entregar **una hoja** que incluya:

1. Explicación del algoritmo Bubble Sort

- ¿Cómo funciona paso a paso?
- ¿Qué hace en cada pasada?
- ¿Por qué se llama “bubble”?

2. Comparación entre las dos versiones

- ¿Qué observaron en Python Tutor?
- ¿Qué diferencias hubo en número de comparaciones?
- ¿Qué diferencias hubo en número de intercambios?
- ¿Qué pasó cuando la lista ya estaba ordenada?

3. Conclusión

- ¿Por qué la variable bandera mejora el rendimiento? ¿En qué casos es más útil?
- ¿Qué aprendieron sobre optimización de algoritmos?